Cool Roofing — Technical README

Cool Roofing Cost & Impact Predictor

Smart India Hackathon 2024 — Smart Infrastructure / Green Buildings Track

Table of Contents

1. Problem
2. Dataset
3. Why Each Tool
4. Data Cleaning Decisions
5. Feature Engineering
6. Why Each ML Algorithm
7. Hyperparameter Choices
8. Why Each Chart
9. Model Performance Values Explained
10. App Architecture
11. File Structure

Problem

Cool roofing contractors in India quote installation costs and energy outcomes from intuition. Building owners commit ₹1L–₹18L with zero pre-installation visibility into: - What will installation cost? - How many degrees will roof temperature drop? - How many kWh/month will be saved? - Will the customer give a high rating (≥4 stars)? - Will the customer refer the contractor?

No pre-install predictive tool existed. We built one.

Dataset

File: `cool_roofing_data.csv` **Rows**: 2,500 (2,472 after cleaning) **Source**: Synthetic dataset based on real cool roofing installation parameters

Columns

Column	Type	Range	Role
`building_id`	int	1–2500	Identifier only
`location_type`	categorical	urban / rural / dense_urban / intermediate	Input feature
`building_size`	float	300–3,000 sq ft	Input feature
`building_age`	float	1–50 years	Input feature
`cool_roofing_quality`	categorical	basic / intermediate / high	Input feature
`roofing_type`	categorical	concrete_flat / concrete_slanted / tiles / sheet	Input feature
`cool_roofing_method`	categorical	basic_coating / tiles / sheet_with_coating / tiles_with_coating / reflective_coating	Input feature
`cool_roofing_cost`	int	₹55,300–₹50,46,600	**Target 1** (regression)
`avg_roof_temperature_before`	float	22.98–85°C (120 = sensor error)	Input feature
`num_ac_units`	float	0–3	Input feature
`avg_units_before`	int	0–500 kWh/month	Input feature
`avg_roof_temperature_after`	float	post-install (not used as input — leakage)	Used to compute target
`avg_units_after`	int	post-install (not used as input)	Used to compute target
`customer_given_rating`	int	1–5	Used to compute target
`customer_referral`	bool	True / False	**Target 5** (classification)

Derived Targets

```
df['temperature_drop'] = df['avg_roof_temperature_before'] -
        df['avg_roof_temperature_after']  # Target 2
df['units_saved']      = df['avg_units_before'] -
        df['avg_units_after']                          # Target 3
df['high_rating']      = (df['customer_given_rating'] >=
        4).astype(int)                          # Target 4
df['referral']         =
        df['customer_referral'].astype(int)
        # Target 5
```

Why these targets? They represent the four questions a building owner asks before installation: How much will it cost? How much cooler? How much cheaper? Will others be happy? Will they recommend?

Why Each Tool

Python

Standard for ML — mature ecosystem (scikit-learn, XGBoost, pandas, numpy). No alternative seriously considered.

pandas + numpy

- `pd.get_dummies()` : one-hot encodes all 4 categorical columns in one call. Alphabetical ordering is deterministic — same column order at train time and inference time. No custom transformer needed.
- `numpy` : vectorised operations for computing targets (subtraction of columns is element-wise without loops).
- `fillna(median())` : robust to outliers unlike mean imputation. Median of building_size ≈ 1,300 sq ft — realistic fill value.

scikit-learn

- `train_test_split` : reproducible splits with `random_state=42` . 70/15/15 chosen because 2,472 rows × 0.15 = 370 test samples — enough for reliable metric estimates.
- `RandomizedSearchCV` : faster than `GridSearchCV` . With 40 iterations over a 5,000+ combination space, finds near-optimal hyperparameters in minutes not hours.
- `confusion_matrix` , `f1_score` , `roc_auc_score` : standard binary classification evaluation metrics used throughout industry.

- `DummyRegressor` / `DummyClassifier`: establishes meaningful baselines. Without a baseline, a 65% accuracy sounds weak. Against a 56% baseline it is a clear improvement.

XGBoost (`xgboost` package)

For installation cost regression — see Why Each ML Algorithm.

joblib

Persists trained sklearn and XGBoost models to disk as `.joblib` files. Faster than `pickle` for large numpy arrays inside model objects. Loads in ~50ms at app startup vs ~2s for retraining.

Streamlit

Single-file web app framework. No HTML/CSS/JS knowledge required. Sidebar, tables, charts, file upload all built-in. Alternative (FastAPI + Jinja2 templates) was built but replaced because Streamlit cuts development time by 60% for an ML demo app.

Plotly Express

Interactive charts with one-line API. `px.scatter`, `px.histogram`, `px.imshow` all accept DataFrames directly. Alternative (matplotlib) produces static images — not suitable for a filterable dashboard.

Matplotlib (notebooks only)

Used in `cool_roofing.ipynb` for EDA and training visualisations. Static output is fine for notebooks — no interactivity needed there.

pptxgenjs

Node.js library for programmatic PowerPoint generation. Chosen because: - Python-pptx cannot duplicate slides (only entry point is `add_slide(layout)`) - pptxgenjs has full theme/master/section/placeholder API - Enables reproducible, version-controlled slide generation

---

Data Cleaning Decisions

120°C Sentinel Values

```
df.loc[df['avg_roof_temperature_before'] == 120,
       'avg_roof_temperature_before'] = np.nan
df.loc[df['avg_roof_temperature_after']  == 120,
       'avg_roof_temperature_after']  = np.nan
```

Why: A rooftop temperature of 120°C is physically impossible — the hottest recorded rooftops in India reach ~85°C. 120 is a sensor error or placeholder sentinel value (32 before, 26 after). Replacing with `NaN` instead of dropping the rows means those 32 rows are still usable for cost, units_saved, rating, and referral tasks that don't depend on temperature.

Missing Values (106 total)

```
X = df[FEATURES].fillna(df[FEATURES].median())
```

Why median not mean: `building_size`, `building_age`, `num_ac_units` all have outliers. Mean of `building_size` is pulled up by a few large buildings. Median (≈1,300 sq ft) is more representative of a typical building.

Why fill at prediction time not training time: Filling at training time leaks test-set statistics into training. We compute medians on the training set and store them in `training_stats.joblib`, then apply those same medians to any unseen batch upload.

No Row Removal for Missing Values

All 2,472 rows with positive `building_size` and `cool_roofing_cost` are retained. Dropping rows for missing `num_ac_units` (31 missing) would waste 31 valid cost/rating/referral training examples.

No Outlier Removal for Cost

Cost ranges from ₹55,300 to ₹50,46,600. These extreme values are genuine — a 3,000 sq ft high-quality installation really does cost ₹50L. Removing them would make the model useless for large commercial buildings.

Feature Engineering

One-Hot Encoding

```
cat_cols = ['location_type', 'cool_roofing_quality', 'roofing_type',
        'cool_roofing_method']
df_encoded = pd.get_dummies(df, columns=cat_cols, drop_first=False)
```

Why `drop_first=False`: Keeping all dummies (not N-1) makes the model more interpretable. XGBoost and Random Forest do not suffer from perfect multicollinearity the way linear models do, so the extra column costs nothing and aids feature importance reading.

Why alphabetical order matters: `pd.get_dummies()` sorts categories alphabetically. The encoded column order at training time must match inference time. Since we always use `pd.get_dummies()` on fresh data, order is always consistent. `encoded_columns.joblib` stores the exact column list for the batch predictor's reindexing step.

Task-Specific Feature Sets

Task	Extra Feature Added	Why
Cost	None (BASE only)	Cost determined by building specs + method/ quality
Temperature Drop	`avg_roof_temperature_before`	Hotter roofs see bigger drops — key signal
Units Saved	`avg_units_before`	Buildings using more electricity save more
High Rating	Both temp + units before	Customer satisfaction correlates with measurable benefit
Referral	Both temp + units before	Same reasoning as rating

Why not use post-installation columns as features? `avg_roof_temperature_after` and `avg_units_after` are only known after installation — using them would be data leakage. The entire goal is prediction before installation happens.

Derived Binary Target for Rating

```
df['high_rating'] = (df['customer_given_rating'] >= 4).astype(int)
```

Why binary not 5-class: Ordinal regression for 5 stars requires ordered logistic regression or complex architectures. Binary classification (satisfied vs not) is actionable — a contractor wants to know "will this customer be happy?" not "will they give 3 or 4 stars?" The 4-star threshold matches industry convention for "positive review."

Why Each ML Algorithm

Task 1 — Installation Cost → XGBoost

Why XGBoost over Random Forest? Cost depends on complex multiplicative interactions: - A 2,000 sq ft building costs roughly 4× a 500 sq ft building (non-linear) - High quality costs 3–4× basic at the same size (non-linear) - Urban buildings have higher labour cost than rural (feature interaction)

XGBoost handles these better because: 1. **Sequential boosting**: Tree N specifically corrects the residuals that Trees 1 to N-1 got wrong. For cost, some high-value outliers are very hard to predict — boosting focuses more on those as training progresses. 2. **Regularisation**: L2 penalty (λ=0.5) and learning rate (lr=0.06) prevent any single tree from overfitting to training data. Random Forest relies only on bagging for variance reduction. 3. **Final $R^2 = 0.9843$** vs Random Forest's $R^2 \approx 0.9797$ (from notebook cell 20) — XGBoost is measurably better here.

Why not Linear Regression? Linear regression assumes the effect of building_size on cost is constant regardless of quality. That is false. A 100 sq ft increase in a high-quality installation costs far more than in a basic installation. XGBoost captures this interaction automatically.

Tasks 2–5 — Random Forest (and Gradient Boosting in final submission model)

Why Random Forest? - Robust to noisy targets (temperature drop, customer rating are inherently variable) - No hyperparameter is as sensitive as XGBoost's learning rate — more stable for targets with less clear signal - `class_weight='balanced'` handles 57:43 and 64:36 class imbalances cleanly - `n_jobs=-1` uses all CPU cores — training 300 trees is fast

Why not XGBoost for classification? XGBoost requires `scale_pos_weight` for imbalance, which is harder to tune. Random Forest's `class_weight='balanced'` automatically reweights samples. For moderate-accuracy classification tasks, the two perform similarly — Random Forest is simpler.

Why `max_depth=12` ? Temperature drop and units saved have many micro-patterns (specific method + quality + size combinations that behave differently). Depth 12 allows trees to find these. Deeper than

12 starts memorising individual rows. Shallower (depth 5) loses important interactions. Verified by checking validation MAE at multiple depth values.

Hyperparameter Choices

XGBoost Cost Model

```
XGBRegressor(
    n_estimators=2000,      # more trees = lower bias; early stopping not
        used, so more is safer
    max_depth=5,            # shallow to prevent overfitting on 1,730
        training rows
    learning_rate=0.06,     # slow enough that 2000 trees have meaningful
        cumulative effect
    subsample=0.9,          # each tree sees 90% of rows — introduces
        variance, reduces overfitting
    colsample_bytree=0.7,   # each tree uses 70% of features — prevents
        dominant features from always splitting
    min_child_weight=1,     # minimum sum of instance weights in a leaf —
        fine-grained splits allowed
    reg_lambda=0.5          # L2 regularisation — penalises large leaf
        weight magnitudes
)
```

Why n_estimators=2000? With lr=0.06, each tree contributes 0.06 × its prediction. 2000 trees × lr=0.06 gives each tree a meaningful but small effect. Fewer trees (e.g. 300) at the same lr underfits — test R^2 was 0.97 vs 0.9843.

Why max_depth=5 for cost but 12 for RF? XGBoost trees with max_depth=5 have 2^5=32 leaf nodes maximum. With 1,730 training rows, depth 5 is already generously expressive. Because XGBoost builds 2,000 of them, each individual tree is kept shallow to prevent any single tree from memorising data. Random Forest builds only 300 trees so each tree can be deeper.

Why subsample=0.9 not 1.0? Stochastic gradient boosting (subsample < 1.0) is empirically more accurate than deterministic boosting on most real datasets. Each tree sees a slightly different view of the data, reducing the risk of all trees exploiting the same quirks.

Why reg_lambda=0.5? L2 regularisation shrinks leaf weights toward zero. Without it (λ=0), extreme predictions for outlier buildings would inflate training accuracy but hurt test performance. λ=0.5 was found via RandomizedSearchCV — values below 0.2 overfit, values above 1.0 underfit.

Random Forest

```
RandomForestRegressor(n_estimators=300, max_depth=12, random_state=42,
        n_jobs=-1)
RandomForestClassifier(n_estimators=300, max_depth=12,
        class_weight='balanced', random_state=42, n_jobs=-1)
```

Why n_estimators=300? Diminishing returns beyond 300 for a 2,400-row dataset. R^2 stabilises at ~300 trees for all tasks. 1000 trees gives negligible improvement but 3× longer training time.

Why class_weight='balanced'? Class distribution for high_rating: 57% not-high, 43% high. Without balancing, RF predicts "not-high" for everything (56.1% baseline accuracy). With `class_weight='balanced'`, effective sample counts are equalised — RF learns to identify the minority positive class. Result: accuracy rises from 56.1% to 65.5% AND F1 rises from ~0.39 to 0.63.

Train/Val/Test Split — 70/15/15

```
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.30,
        random_state=42)
X_val, X_test, y_val, y_test     = train_test_split(X_temp, y_temp,
        test_size=0.50, random_state=42)
```

Why 70/15/15? With 2,472 rows, 70% = 1,730 training samples. 15% = ~370 validation and test samples each. 370 samples is the minimum for reliable metric estimates for both regression (MAE, R^2) and classification (accuracy, F1, AUC). An 80/10/10 split would give only ~247 test samples — too few for stable AUC estimation.

Why random_state=42? Any fixed seed produces a reproducible split. 42 is convention (no mathematical significance). Fixed seed ensures results in the notebook match results in the app.

Why Each Chart

Chart 1 — Target Distributions (3 histograms)

Purpose: Understand what we are predicting before modelling.

- **Cost histogram**: Right-skewed. Mean ₹10.9L ≠ median ₹8.6L. This skew tells us linear regression will underperform — the high-cost tail is real signal, not noise. XGBoost handles skewed targets better than linear models.
- **Temperature drop histogram**: Near-normal, centred at 8.1°C. Low skew means Random Forest regression is appropriate — no transformation needed.
- **Units saved histogram**: Right-skewed (like cost). Buildings with high baseline usage save a lot; zero-AC buildings save nothing. The avg_units_before feature directly captures this.

Chart 2 — Categorical Distributions (bar charts)

Purpose: Check for class imbalance in input categories that might cause model bias.

- **Roof type**: Balanced across 4 types. No type is so rare that the model fails to learn it.
- **Cool roofing method**: 5 methods roughly balanced at ~400–550 each. No method is underrepresented.

Chart 3 — Cost by Segment (horizontal bar charts)

Purpose: Confirm that categorical features have genuine predictive value before encoding them.

- **By location type**: Dense urban (₹14L avg) vs rural (₹7L avg) — 2× difference confirms location matters.
- **By quality tier**: High (₹18L avg) vs basic (₹5L avg) — 3.6× difference. Quality is the strongest single predictor of cost, which the feature importance chart (Chart 4) confirms.

Chart 4 — Feature Importances (5 horizontal bar charts)

Purpose: Understand what each model is actually learning. Used to verify no data leakage and that predictions are driven by sensible features.

- **Cost**: quality_high has highest importance. building_size second. Makes physical sense.
- **Temperature drop**: avg_roof_temperature_before dominant (importance ~0.40). Hotter pre-install = bigger drop. Makes physical sense.
- **Units saved**: avg_units_before dominant (importance ~0.44). More baseline usage = more to save. Makes physical sense.
- **High rating**: avg_units_before, num_ac_units — AC-heavy buildings benefit more visibly, making customers happier.
- **Referral**: Similar pattern to rating.

If the top features did not make physical sense, we would suspect a data leakage bug.

Chart 5 — Confusion Matrices (2 heatmaps)

Purpose: Understand not just accuracy but which type of error the model makes.

- **High rating**: The matrix shows more True Negatives than True Positives — the model is conservative. Acceptable: better to not falsely promise high satisfaction.
- **Referral**: Fewer True Positives than we'd want — referral is harder to predict from physical features. The notes in the PPT explain why this is expected.

Blues colormap for rating, Greens for referral — visually distinct at a glance.

Chart 6 — Model Performance Summary (grouped bar charts)

Purpose: One slide showing all 5 model performances side by side vs their baselines.

- **Left chart**: Regression R^2 — shows how much better our models are vs always predicting the mean (R^2=0 baseline). Cost 0.984, Temp 0.935, Units 0.944.
- **Right chart**: Classification accuracy — shows model (65.5%, 62.0%) vs stratified baseline (56.1%, 57.1%).

The baseline bar next to each model bar makes the improvement immediately obvious without mental arithmetic.

Chart 7 — Cost Scatter (building_size vs cost, coloured by quality)

Purpose: Visual confirmation that the non-linear interaction between size and quality is the dominant cost signal.

The clear vertical separation of colour bands (green=basic at bottom, navy=high at top) confirms the model is learning real physics, not noise. Overlap within bands shows that size adds a second dimension of variation within each quality tier.

Chart 8 — Customer Satisfaction (rating bar + referral pie)

Purpose: Show the raw distribution of the two classification targets and motivate the class balancing decision.

- **Rating bar**: 5 bars coloured by satisfaction level. Shows 65% positive (4–5 stars) — justifies the ≥4 threshold for binary classification.
- **Referral pie**: 64/36 split visible immediately — confirms class imbalance that needs `class_weight='balanced'`.

Model Performance Values Explained

R^2 = 0.9843 (Installation Cost)

R^2 measures what fraction of variance the model explains. 0.9843 means 98.4% of cost variation across buildings is captured by our 9 features. The remaining 1.6% is due to factors not in the dataset (local contractor pricing, material quality variation, negotiation).

Baseline R^2 = 0.0 (predicting the mean for every building).

MAE = ₹69,426 (Installation Cost)

Mean Absolute Error: on average, predictions are ₹69,426 away from actual cost. For a project averaging ₹10.9 lakhs, this is a 6.4% average error rate. The baseline MAE was ₹6,08,878 — our model is 8.8× better.

MAE = 0.463°C (Temperature Drop)

Average prediction error of less than half a degree Celsius. For a target that ranges from 2°C to 15°C drop, this is excellent precision. Baseline MAE was 1.67°C — our model is 3.6× better.

MAE = 5.41 kWh (Units Saved)

Average prediction error of 5.41 kWh per month. At ₹7/kWh tariff, this error corresponds to ₹38/month uncertainty in savings forecast — acceptable for financial planning. Baseline MAE was 39.74 kWh.

Accuracy = 65.5% (High Rating)

65.5% of test buildings are correctly classified as high or low rating. Baseline = 56.1% (stratified random). Improvement = +9.4 percentage points. ROC-AUC = 0.728 confirms the model has real discriminative ability (AUC = 0.5 is random, 1.0 is perfect).

Accuracy = 62.0% (Referral)

62.0% vs baseline 57.1%. +4.9 percentage points improvement. F1 = 0.505 — the model finds genuine referral signal. Lower than rating because referral depends on social and network factors not captured in the dataset.

Row 22 Example

Building at index 20 in cleaned DataFrame (displayed as row 22 in 1-indexed notebook): - building_size = 900 sq ft, urban, age 11, 1 AC unit, concrete_slanted, tiles, basic quality - Predicted cost: ₹3,10,742 - Actual cost: ₹3,13,700 - Error: 0.94% — under 1%

This specific example was used throughout the presentation to make predictions concrete. A < 1% error on a single example does not mean average error is < 1% (average MAE is ₹69,426 = 6.4% of mean cost), but it illustrates what a good prediction looks like.

App Architecture

encode_batch_masked() — Why Vectorised

```
def encode_batch_masked(df_input, all_cols, exclude_cols):
    dummies = pd.get_dummies(df_input, columns=CAT_COLS, drop_first=False)
    dummies = dummies.reindex(columns=all_cols, fill_value=0)
    for col in exclude_cols:
        if col in dummies.columns:
            dummies[col] = 0.0
    return dummies
```

This function encodes an entire DataFrame at once — no Python `for row in df` loop. `pd.get_dummies()` + `reindex()` are numpy-backed C operations. 2,500 rows takes 0.23 seconds. A row-by-row loop would take ~2.3 seconds (10× slower).

Why `reindex(fill_value=0)` ? A batch CSV might not contain all 4 categorical columns or might use different category values. `reindex` with the full list of expected columns fills any missing dummies with 0 — treating them as "not in that category". This is safer than raising an error and stops unknown categories from crashing the app.

Feature Masking (not retraining)

```
dummies[col] = 0.0  # for excluded features
```

When a user excludes a feature from batch prediction, we zero it out in the encoded matrix rather than retraining a model without that feature. This is correct because one-hot encoded columns default to 0 = "not in this category." For numeric features, setting to 0 biases predictions toward zero-AC-unit buildings (if AC is excluded), but it avoids the 10+ second retraining cost for interactive use.

training_stats.joblib — Why Save Medians

At training time, medians are computed on the training set. At inference time (batch upload), those same training medians are used to fill missing values — not the medians of the uploaded batch. This prevents test data from leaking information about itself into the preprocessing step.

Recommender — Weighted Score Formula

```
score = (
    -0.25 * norm(cost) +
    +0.25 * norm(temp_drop) +
    +0.25 * norm(units_saved) +
    +0.15 * prob(high_rating) +
```

```
    +0.10 * prob(referral)
)
```

Weights sum to 1.0. Cost has negative weight (lower cost = better). All values are min-max normalised across the 15 combinations before weighting — so a ₹3L cost difference and a 5°C temperature difference are on the same 0–1 scale.

Why these weights? 0.25 each to cost, temp, units because these are the three objectively measurable outcomes a building owner cares about equally. 0.15 to rating and 0.10 to referral because they are proxies for future business value, not direct owner outcomes.

File Structure

```
submission/
├── app.py                      # Main Streamlit application (5 pages)
├── cool_roofing_data.csv       # Dataset (2,500 rows)
├── cool_roofing.ipynb          # Full EDA + model training notebook
├── models/
│   ├── model_cost.joblib           # XGBoost trained model
│   ├── model_temp_drop.joblib      # RF trained model
│   ├── model_units_saved.joblib    # RF trained model
│   ├── model_high_rating.joblib    # RF classifier
│   ├── model_referral.joblib       # RF classifier
│   ├── features_*.joblib           # Feature column lists per task
│   ├── encoded_columns.joblib      # Full one-hot column list
│   └── training_stats.joblib       # Training-set medians for inference
├── cool_roofing_SIH.pptx       # SIH presentation (12 slides)
├── cool_roofing_hackathon.pptx # Alternate dark-theme presentation
└── README.md                   # This file
```

Running the App

```
pip install streamlit plotly pandas numpy scikit-learn xgboost joblib
streamlit run app.py
```

App starts on `http://localhost:8501`

Key Numbers at a Glance

Metric	Value
Dataset rows (after cleaning)	2,472
Training rows	1,730
Validation rows	371
Test rows	371
Cost model R^2	0.9843
Cost model MAE	₹69,426
Cost baseline MAE	₹6,08,878
Temp drop MAE	0.463°C
Units saved MAE	5.41 kWh
High rating accuracy	65.5%
High rating AUC	0.728
Referral accuracy	62.0%
Referral F1	0.505
Batch inference speed	2,500 rows in < 0.3 s
Row 22 prediction error	0.94%